

Assignment II: Scheduling Policy Demonstration Program

313512009 施丞蔚

I. Describe how you implemented the program in detail. (10%)

1. 在 **Docker** 裡建立作業目錄, 並放入助教提供的檔案

```
root@ecb5a3b62549:/home/ubuntu# mkdir hw2
```

```
root@ecb5a3b62549:/home/ubuntu/hw2# ls
sched_demo  sched_test.sh
```

2. 放入 `sched_demo_313512009.c` 與 `Makefile` (或用 `cat` 寫)

`sched_demo_313512009.c`

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <getopt.h>
#include <errno.h>
#include <unistd.h>
#include <sched.h>
#include <time.h>

typedef enum { POL_NORMAL = 0, POL_FIFO = 1 } pol_t;

typedef struct {
    int id;
    double busy_sec;
    pol_t policy;
    int rt_prio; // -1 for NORMAL
} thread_arg_t;

static int g_num_threads = 0;
static thread_arg_t *g_args = NULL;
static pthread_t *g_tids = NULL;
static pthread_barrier_t g_barrier;

static int parse_csv_policies(const char *s, pol_t *out, int n)
```

```

{
    char *dup = strdup(s);
    if (!dup) return -1;
    int i = 0;
    char *tok, *save = NULL;
    for (tok = strtok_r(dup, ",", &save); tok && i < n; tok =
strtok_r(NULL, ",", &save), ++i) {
        if (strcasecmp(tok, "NORMAL") == 0 || strcasecmp(tok,
"SCHED_NORMAL") == 0 || strcasecmp(tok, "OTHER") == 0)
            out[i] = POL_NORMAL;
        else if (strcasecmp(tok, "FIFO") == 0 || strcasecmp(tok,
"SCHED_FIFO") == 0)
            out[i] = POL_FIFO;
        else { free(dup); return -2; }
    }
    free(dup);
    return (i == n) ? 0 : -3;
}

static int parse_csv_ints(const char *s, int *out, int n) {
    char *dup = strdup(s);
    if (!dup) return -1;
    int i = 0;
    char *tok, *save = NULL;
    for (tok = strtok_r(dup, ",", &save); tok && i < n; tok =
strtok_r(NULL, ",", &save), ++i) {
        out[i] = atoi(tok);
    }
    free(dup);
    return (i == n) ? 0 : -2;
}

static void busy_wait_seconds(double secs) {
    // Count per-thread CPU time (excludes preempted time).
    struct timespec t0, t1;
    double acc = 0.0;
    clock_gettime(CLOCK_THREAD_CPUTIME_ID, &t0);
    while (acc < secs) {
        for (volatile int i = 0; i < 1000; ++i) { }
        clock_gettime(CLOCK_THREAD_CPUTIME_ID, &t1);
        acc = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec)
/ 1e9;
    }
}

```

```

    }
}

static void *worker(void *argp) {
    thread_arg_t *arg = (thread_arg_t *)argp;
    // 1) 同步同時開跑
    pthread_barrier_wait(&g_barrier);
    // 2) 三回合：列印 → 忙等
    for (int i = 0; i < 3; ++i) {
        printf("Thread %d is running\n", arg->id);
        fflush(stdout);
        busy_wait_seconds(arg->busy_sec);
    }
    return NULL;
}

static void pin_self_cpu0() {
    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(0, &set);
    sched_setaffinity(0, sizeof(set), &set);
}

static void usage(const char *prog) {
    fprintf(stderr,
        "Usage: %s -n <num_threads> -t <time_wait> -s <policies>
-p <priorities>\n"
        "   policies   : CSV of NORMAL/FIFO (len = n)\n"
        "   priorities : CSV of ints (len = n), use -1 for
NORMAL\n", prog);
}

int main(int argc, char **argv) {
    setvbuf(stdout, NULL, _IOLBF, 0); // line-buffer for QEMU

    int opt, n = -1;
    double t = -1.0;
    const char *s_pol = NULL, *s_pri = NULL;

    while ((opt = getopt(argc, argv, "n:t:s:p:")) != -1) {
        switch (opt) {
            case 'n': n = atoi(optarg); break;

```

```

        case 't': t = atof(optarg); break;
        case 's': s_pol = optarg; break;
        case 'p': s_pri = optarg; break;
        default: usage(argv[0]); return 1;
    }
}

if (n <= 0 || t <= 0.0 || !s_pol || !s_pri) { usage(argv[0]);
return 1; }

g_num_threads = n;
g_args = calloc(n, sizeof(*g_args));
g_tids = calloc(n, sizeof(*g_tids));
pol_t *pols = calloc(n, sizeof(*pols));
int *pris = calloc(n, sizeof(*pris));
if (!g_args || !g_tids || !pols || !pris) { perror("alloc");
return 1; }

if (parse_csv_policies(s_pol, pols, n) != 0) {
fprintf(stderr, "Bad -s\n"); return 1; }
if (parse_csv_ints(s_pri, pris, n) != 0) { fprintf(stderr,
"Bad -p\n"); return 1; }

pthread_attr_t *attrs = calloc(n, sizeof(*attrs));
if (!attrs) { perror("alloc attrs"); return 1; }

// 主執行緒綁 CPU0; 所有 worker 也會設 affinity 到 CPU0
pin_self_cpu0();

// 同步點: n 個 worker
pthread_barrier_init(&g_barrier, NULL, n);

for (int i = 0; i < n; ++i) {
    g_args[i].id = i;
    g_args[i].busy_sec = t;
    g_args[i].policy = pols[i];
    g_args[i].rt_prio = pris[i];

    pthread_attr_init(&attrs[i]);
    // 用顯式排程參數 (不繼承)
    pthread_attr_setinheritsched(&attrs[i],
PTHREAD_EXPLICIT_SCHED);

```

```

        int policy = (pols[i] == POL_FIFO) ? SCHED_FIFO :
        SCHED_OTHER;
        pthread_attr_setschedpolicy(&attrs[i], policy);

        struct sched_param sp = {0};
        if (policy == SCHED_FIFO) {
            sp.sched_priority = (g_args[i].rt_prio < 1) ? 1 :
            g_args[i].rt_prio; // 1..99
        } else {
            sp.sched_priority = 0;
        }
        pthread_attr_setschedparam(&attrs[i], &sp);

        if (pthread_create(&g_tids[i], &attrs[i], worker,
        &g_args[i]) != 0) {
            perror("pthread_create"); return 1;
        }

        // 每個 worker 綁到 CPU0
        cpu_set_t set;
        CPU_ZERO(&set);
        CPU_SET(0, &set);
        if (pthread_setaffinity_np(g_tids[i], sizeof(set), &set)
        != 0) {
            perror("pthread_setaffinity_np"); return 1;
        }
    }

    for (int i = 0; i < n; ++i) {
        pthread_join(g_tids[i], NULL);
        pthread_attr_destroy(&attrs[i]);
    }
    pthread_barrier_destroy(&g_barrier);

    free(attrs);
    free(pols);
    free(pris);
    free(g_args);
    free(g_tids);
    return 0;
}

```

Makefile

```
STUDENT ?= 313512009
TARGET  ?= sched_demo_$(STUDENT)

CC      ?= riscv64-linux-gnu-gcc
CFLAGS  ?= -O2 -Wall -Wextra -Werror -pedantic
LDFLAGS ?= -static -pthread

SRC := sched_demo_$(STUDENT).c
OBJ := $(SRC:.c=.o)

.PHONY: all clean
all: $(TARGET)

$(TARGET): $(OBJ)
    $(CC) $(OBJ) -o $@ $(LDFLAGS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(TARGET) $(OBJ)
```

3. 交叉編譯為 RISC-V

```
root@ecb5a3b62549:/home/ubuntu/hw2# make CC=riscv64-linux-gnu-gcc
riscv64-linux-gnu-gcc -O2 -Wall -Wextra -Werror -pedantic -c sched_demo_313512009.c -o sched_demo_313512009.o
riscv64-linux-gnu-gcc sched_demo_313512009.o -o sched_demo_313512009 -static -pthread
root@ecb5a3b62549:/home/ubuntu/hw2# ls
Makefile sched_demo sched_demo_313512009.c sched_demo_313512009.o sched_test.sh
```

註：

由於Makefile裡面寫：`CC ?= riscv64-linux-gnu-gcc`，所以終端指令用的是make
`CC=riscv64-linux-gnu-gcc`，或把Makefile裡面該行程式碼改為 `CC := riscv64-linux-gnu-gc`，兩者皆是強制使用交叉編譯

4. 複製到 initramfs

```
root@ecb5a3b62549:/home/ubuntu/hw2# cp /home/ubuntu/hw2/sched_demo_313512009 /home/ubuntu/initramfs/
root@ecb5a3b62549:/home/ubuntu/hw2# cp /home/ubuntu/hw2/sched_demo /home/ubuntu/initramfs/
root@ecb5a3b62549:/home/ubuntu/hw2# cp /home/ubuntu/hw2/sched_test.sh /home/ubuntu/initramfs/
```

5. 重新封裝

```
root@ecb5a3b62549:/home/ubuntu/hw2# cd /home/ubuntu/initramfs
root@ecb5a3b62549:/home/ubuntu/initramfs# find . | cpio -o -H newc | gzip > ../initramfs.cpio.gz
8751 blocks
root@ecb5a3b62549:/home/ubuntu/initramfs# cd ..
```

6. 啟動 QEMU

[illegible]

```
~ # uname -a
Linux (none) 6.1.0-os-313512009+ #3 SMP Fri Oct 17 09:39:47 UTC 2025 riscv64 GNU/Linux
~ # cat /proc/version
Linux version 6.1.0-os-313512009+ (ubuntu@ecb5a3b62549) (riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #3 SMP Fri Oct 17 09:39:47 UTC 2025
```

7. 執行示範程式之前, 在 **QEMU** 中執行下列命令以停用 **RT** 節流

```
~ # echo -1 > /proc/sys/kernel/sched_rt_runtime_us
```

8. 在 **QEMU** 內測試

```
~ # chmod +x /sched_demo /sched_demo_313512009 /sched_test.sh
~ # ./sched_test.sh /sched_demo /sched_demo_313512009
Running testcase 1: /sched_demo -n 1 -t 0.5 -s NORMAL -p -1
Result: Success!
Running testcase 2: /sched_demo -n 2 -t 0.5 -s FIFO,FIFO -p 10,20
Result: Success!
Running testcase 3: /sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
Result: Success!
```

```
~ # ./sched_demo -n 4 -t 0.5 -s NORMAL,FIFO,NORMAL,FIFO -p -1,10,-1,30
Thread 3 is running
Thread 3 is running
Thread 3 is running
Thread 1 is running
Thread 1 is running
Thread 1 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
```

II. Describe the results of `./sched_demo -n 3 -t 1.0 -s NORMAL, FIFO, FIFO -p -1, 10, 30` and what causes that. (10%)

結果(輸出順序的型態):

在同一顆 CPU 上, 三個執行緒的輸出會呈現三段式

1. 最高優先權的 **FIFO(prio=30)** 連續印完 3 行:
Thread 2 is running ×3
2. 接著較低優先權的 **FIFO(prio=10)** 連續印完 3 行:
Thread 1 is running ×3
3. 最後才輪到 **NORMAL(CFS)** 把 3 行印完:
Thread 0 is running ×3

(thread 編號以建立順序 0,1,2 為例, 對應 NORMAL, FIFO(10), FIFO(30))

原因(排程機制):

- **SCHED_FIFO** 為搶佔式即時排程、無 time slice; 同 CPU 上只要就緒, 它會壓過 **SCHED_OTHER**(NORMAL)。
- 且 較高優先(30) 會壓過 較低優先(10) 的 FIFO。
- 你的 worker 不會主動讓賢(yield)也不會阻塞; 因此 **FIFO(30)** 一口氣跑完三回合(每回合: 印字 → 忙等), 才換 **FIFO(10)**, 最後 NORMAL 才有機會執行。

```
Running testcase 3: ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
Result: Success!
```

III. Describe the results of `./sched_demo -n 4 -t 0.5 -s NORMAL, FIFO, NORMAL, FIFO -p -1, 10, -1, 30`, and what causes that. (10%)

結果(輸出順序的型態):
四個執行緒在單核上, 輸出會是

1. **FIFO(prio=30)** (thread 3) 先 連續 3 行: **Thread 3 is running** ×3
2. 再來 **FIFO(prio=10)** (thread 1) 連續 3 行: **Thread 1 is running** ×3
3. 兩個 **NORMAL** (threads 0 與 2) 最後才執行, 它們彼此之間會由 CFS 時間片公平交替, 整體上會把各自的 3 行印完, 但可能互相穿插(例如 0、2、0、2... 的型態; 精確交錯次序可能隨執行時機略有差異)。

原因(排程機制):

- 與上一題相同:兩個 FIFO 先於 NORMAL, 且 prio=30 完整跑完三回合後才換 prio=10。
- NORMAL 彼此之間由 CFS 公平分配時間片, 因此兩個 NORMAL 的輸出行通常會 交替出現, 直到各自完成三回合。

```
~ # /sched_demo -n 4 -t 0.5 -s NORMAL,FIFO,NORMAL,FIFO -p -1,10,-1,30
Thread 3 is running
Thread 3 is running
Thread 3 is running
Thread 1 is running
Thread 1 is running
Thread 1 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
```

IV. Describe how did you implement n-second-busy-waiting? (10%)

以每個執行緒的 CPU time 來計時:使用

`clock_gettime(CLOCK_THREAD_CPUTIME_ID, ...)` 讀取 該執行緒自己在 CPU 上真實消耗的時間。

迴圈中做極短的無副作用運算(如小計數), 反覆量測「目前 CPU time - 起始 CPU time」, 直到累積達到指定的 `t` 秒。

關鍵程式架構:

```
static void busy_wait_seconds(double secs) {
    // Count per-thread CPU time (excludes preempted time).
    struct timespec t0, t1;
    double acc = 0.0;
    clock_gettime(CLOCK_THREAD_CPUTIME_ID, &t0);
    while (acc < secs) {
        for (volatile int i = 0; i < 1000; ++i) { }
        clock_gettime(CLOCK_THREAD_CPUTIME_ID, &t1);
        acc = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
    }
}
```

為什麼要用 thread CPU time ?

- 它不包含被搶佔期間。若用牆鐘時間(wall-clock), 低優先緒被 RT 搶走 CPU 時, 忙等會被「拉長」, 造成每回合用時不準、輸出順序受外在搶占嚴重干擾。
- 用 thread CPU time 可確保每回合的「有效運算」接近指定秒數, 方便觀察排程政策/優先權本身造成的順序差異。

V.What does the `kernel.sched_rt_runtime_us` effect? If this setting is changed (eg. 500000, 950000, 1000000), what will happen?(10%)

它的影響:

- Linux 的 RT 節流參數:限制每顆 CPU 在一個週期(預設 `sched_rt_period_us` 通常為 1,000,000 μ s)內, RT (SCHED_FIFO/SCHED_RR) 可用的總 CPU 時間上限。
- 值為 `-1` 表「停用節流」:RT thread 幾乎可長時間佔用 CPU, 不會被強制讓出。
- 值為正整數時(例如 500000):代表 RT threads 在每個週期最多只能用到對應的 μ s, 超過就會被暫時 throttled, 改由 CFS 任務(NORMAL)上 CPU。

改為 500000 / 950000 / 1000000 :

- 500000 (50%):RT 每個週期最多用半個 CPU;你的兩個 FIFO 執行緒在長時間忙等時會被「插斷」→ 被節流後, NORMAL 有機會在中途上 CPU。輸出上會看到 NORMAL 的列印行穿插在 FIFO 的行之間, 且總執行時間會拉長。
- 950000 (95%):RT 幾乎壟斷, 但仍會偶爾被節流一下; NORMAL 只有極短的插縫可以執行, 穿插行變少、延遲更長。
- 1000000 ($\approx 100\% = 1$ 週期):等同允許 RT 佔滿整個週期(實務上接近停用節流), 行為趨近 `-1`, NORMAL 幾乎要等 RT 全部跑完才有較完整的時間片。